

MACHINE TRANSLATION AND IMPLEMENTATION OF TRANSFORMERS

*Submitted in partial fulfilment of
the credit requirements for the degree of
Bachelor of Technology with Honours
in Computer Science and Engineering
(Artificial Intelligence and Machine Learning)*

Padavinangady Nakul Bhat

Reg. No. 230962212
School of Computer Engineering
Manipal Institute of Technology

Supervised by

Dr. Manoj T

July–December 2025

Abstract

This report covers the topics presented in the “Introductions to Large Language Models” course taught by Tanmoy Chakraborty and Soumen Chakrabarti, available on NPTEL. As a graded assignment, the report follows a set of seven questions, and accompanies a code base available in the repository linked below. Each question will be quoted in full, followed by explanations and code listings where appropriate.

Note To Readers

A repository accompanying this report is available at the author’s GitHub homepage. It is accessible to the public and can be navigated to by clicking or scanning the QR Code on the right.

The repository contains source code used for the development of models described in this report in the `code/` directory. The repository also contains the $\text{\LaTeX}$ source code used for the compilation of this report in the `report/` directory. A `flake.nix` and `flake.lock` have also been provided for readers familiar with Nix to reproduce the development environment.



QR Code linking to
accompanying
repository

Work Attribution

All work for this project has been carried out without any AI assistance or code/text generation. A work history is provided at the end of the report as a proof of work. This work was solely developed by me, P Nakul Bhat, without any undeclared assistance. All sources used for references have been declared in the report, and are also available as a bibliography file in `report/` directory.

This work is licensed under a Creative Commons “Attribution-ShareAlike 4.0 International” license.



```

1 class Encoder(nn.Module):
2     def __init__(self, input_size, hidden_size, dropout_p = 0.1):
3         super(Encoder, self).__init__()
4         self.hidden_size = hidden_size
5         self.embedding = nn.Embedding(input_size, hidden_size)
6         self.gru = nn.GRU(hidden_size, hidden_size, batch_first=True)
7         self.dropout = nn.Dropout(dropout_p)
8
9     def forward(self, input):
10        embedded = self.dropout(self.embedding(input))
11        output, hidden = self.gru(embedded)
12        return output, hidden

```

Listing 1: Encoder of the Seq2Seq Model

1 Sequence-to-Sequence Machine Translation Model

Implement a Transformer-based sequence-to-sequence model in PyTorch for machine translation (English $\rightarrow$ French). Explain the encoder-decoder structure, attention mechanism, and training process. Provide code snippets and discuss evaluation metrics such as BLEU score

The full working implementation is available in `code/seq2seq.py`. Only important parts of the code are discussed below to provide an illustration of the working. The code largely follows the flow outlined in the PyTorch Documentation [3]. The dataset was downloaded from Tatoeba [4], and preprocessed before usage. Preprocessing included extracting pairs of English sentences with French translations, and converting Unicode strings into ASCII strings.

First, we define a class to help build a vocabulary and get one-hot encoded vectors. The `class Lang` is responsible for the ingestion of sentences and the storage of dictionaries holding the indices of words present in the language. A simple tokenizer is used, and tokens are split by whitespaces.

The encoder-decoder architecture is one of three possible transformer architectures [1]. It is especially suited for summarization and machine translation as there is no restriction on both the input and output length. It uses the encoder part to calculate a dense context vector that effectively captures all the data from the input vector. The encoder we have used is an RNN with a Gated Recurrent Unit that allows it to have better context and memory. The encoder code is given in Listing 1

The decoder part of the transformer consists of another RNN-GRU, and uses attention along with already generated tokens to get a probability distribution of possible continuations. The decoder can work in two ways, by using previous tokens as mentioned earlier, or by “teacher-forcing” where we can force the next step of attention to use tokens provided by a ground truth. Teacher-forcing allows a model to train faster

and make less mistakes by not allowing mistakes to propagate far into the model. Code for the decoder can be found in Listing 2

```
1 class AttnDecoderRNN(nn.Module):
2     def __init__(self, hidden_size, output_size, dropout_p=0.1):
3         super(AttnDecoderRNN, self).__init__()
4         self.embedding = nn.Embedding(output_size, hidden_size)
5         self.attention = BahdanauAttention(hidden_size)
6         self.gru = nn.GRU(2 * hidden_size, hidden_size,
7             ↪ batch_first=True)
8         self.out = nn.Linear(hidden_size, output_size)
9         self.dropout = nn.Dropout(dropout_p)
10
11     def forward(self, encoder_outputs, encoder_hidden,
12         ↪ target_tensor=None):
13         batch_size = encoder_outputs.size(0)
14         decoder_input = torch.empty(batch_size, 1, dtype=torch.long,
15             ↪ device=DEVICE).fill_(SOS_TOKEN)
16         decoder_hidden = encoder_hidden
17         decoder_outputs = []
18         attentions = []
19
20         for i in range(MAX_STEPS):
21             decoder_output, decoder_hidden, attn_weights =
22                 ↪ self.forward_step(
23                 decoder_input, decoder_hidden, encoder_outputs
24             )
25             decoder_outputs.append(decoder_output)
26             attentions.append(attn_weights)
27
28             if target_tensor is not None:
29                 # Teacher forcing: Feed the target as the next input
30                 decoder_input = target_tensor[:, i].unsqueeze(1) #
31                 ↪ Teacher forcing
32             else:
33                 # Without teacher forcing: use its own predictions as
34                 ↪ the next input
35                 _, topi = decoder_output.topk(1)
36                 decoder_input = topi.squeeze(-1).detach() # detach from
37                 ↪ history as input
38
39         decoder_outputs = torch.cat(decoder_outputs, dim=1)
40         decoder_outputs =
41             ↪ torch.nn.functional.log_softmax(decoder_outputs, dim=-1)
```

```

34     attentions = torch.cat(attentions, dim=1)
35
36     return decoder_outputs, decoder_hidden, attentions
37
38
39 def forward_step(self, input, hidden, encoder_outputs):
40     embedded = self.dropout(self.embedding(input))
41
42     query = hidden.permute(1, 0, 2)
43     context, attn_weights = self.attention(query, encoder_outputs)
44     input_gru = torch.cat((embedded, context), dim=2)
45
46     output, hidden = self.gru(input_gru, hidden)
47     output = self.out(output)
48
49     return output, hidden, attn_weights

```

Listing 2: Attention Decoder of the Seq2Seq Model

Attention is a mechanism to control the tokens that a model considers relevant and important. Attention allows a decoder access to all the states of the encoder, and selectively picks the most important ones.

For every generated token, attention calculates an alignment score, called the *attention weight*. Instead of compressing all information from the encoder into a single fixed-length vector, attention computes a weighted combination of all encoder hidden states. This allows the decoder to dynamically focus on different parts of the input sequence at each decoding step.

In this work, we specifically use *Bahdanau attention* [2], which is an additive attention mechanism. Given the encoder hidden states h_i and the current decoder hidden state s_t , an alignment score is computed as

$$e_{t,i} = v_a^\top \tanh(W_a s_t + U_a h_i), \quad (1)$$

where W_a , U_a , and v_a are learnable parameters. These scores are normalized using a softmax function to obtain attention weights

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}. \quad (2)$$

The context vector c_t is then computed as a weighted sum of encoder hidden states

$$c_t = \sum_i \alpha_{t,i} h_i. \quad (3)$$

This context vector is concatenated with the decoder hidden state and passed to the output layer to generate the next token. The attention implementation used in this project is shown in Listing 3.

```

1 class BahdanauAttention(nn.Module):
2     def __init__(self, hidden_size):
3         super(BahdanauAttention, self).__init__()
4         self.Wa = nn.Linear(hidden_size, hidden_size)
5         self.Ua = nn.Linear(hidden_size, hidden_size)
6         self.Va = nn.Linear(hidden_size, 1)
7
8     def forward(self, query, keys):
9         scores = self.Va(torch.tanh(self.Wa(query) + self.Ua(keys)))
10        scores = scores.squeeze(2).unsqueeze(1)
11
12        weights = torch.nn.functional.softmax(scores, dim=-1)
13        context = torch.bmm(weights, keys)
14
15    return context, weights

```

Listing 3: Bahdanau Attention used in the Seq2Seq Model

The training process consists of minimizing the negative log-likelihood loss between the predicted output sequence and the ground truth translation. Given a target sentence $y = (y_1, \dots, y_T)$, the loss is defined as

$$\mathcal{L} = - \sum_{t=1}^T \log p(y_t \mid y_{<t}, x), \quad (4)$$

where x is the input English sentence. The model is trained using the Adam optimizer, and gradients are computed using backpropagation through time. Teacher-forcing is applied with a fixed probability, allowing the decoder to alternate between using its own predictions and ground-truth tokens during training. This improves convergence speed and stability.

During evaluation, the model is switched to inference mode and generates translations autoregressively without teacher-forcing. To quantitatively assess translation quality, we use the BLEU (Bilingual Evaluation Understudy) score, which measures the overlap of n -grams between the generated translation and a reference translation. BLEU is defined as

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right), \quad (5)$$

where p_n denotes the precision of n -grams, w_n are uniform weights, and BP is a brevity penalty that discourages overly short translations. BLEU provides a standardized and widely-used metric for comparing machine translation systems, and is well-suited for evaluating the English-to-French task considered here.

2 Transformers and Multi-Head Attention

Explain the architecture of a Transformer model and implement a minimal Transformer encoder using PyTorch. Highlight the role of multi-head attention and position-wise feed-forward networks. Provide code snippets to support your answer.

The Transformer architecture was introduced to address the limitations of recurrent models in sequence modeling tasks, most notably their sequential computation and difficulty in capturing long-range dependencies. Unlike RNN-based models, Transformers rely entirely on attention mechanisms and dispense with recurrence and convolution altogether [5]. This allows for full parallelization during training and more efficient modeling of global context.

A Transformer is composed of stacked encoder and decoder blocks. In this section, we focus on a minimal Transformer encoder, which is sufficient to illustrate the core architectural components. Each encoder block consists of two main sub-layers: a multi-head self-attention mechanism and a position-wise feed-forward network. Both sub-layers are wrapped with residual connections followed by layer normalization.

Self-attention enables the model to compute contextualized representations of tokens by attending to all other tokens in the input sequence. Given an input sequence represented as a matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$, self-attention projects the input into queries, keys, and values using learned linear transformations

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (6)$$

The attention output is then computed using scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V, \quad (7)$$

where d_k is the dimensionality of the key vectors. The scaling factor stabilizes gradients for large values of d_k .

Multi-head attention extends this idea by running several attention operations in parallel. Each head attends to the input sequence using different learned projections, allowing the model to capture multiple types of relationships simultaneously. Formally, the output of multi-head attention is given by

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \quad (8)$$

where each head is an independent attention computation. This mechanism significantly increases the expressive power of the model without a large computational overhead.

Following the attention sub-layer, each encoder block contains a position-wise feed-forward network. This network is applied independently and identically to each position in the sequence, and consists of two linear transformations with a non-linear activation function in between

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2. \quad (9)$$

The feed-forward network increases the representational capacity of the model and introduces non-linearity after the attention operation.

```

1 class TransformerEncoderLayer(nn.Module):
2     def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
3         super().__init__()
4         self.self_attn = nn.MultiheadAttention(d_model, num_heads,
5         ↪ dropout=dropout)
6         self.ffn = nn.Sequential(
7             nn.Linear(d_model, d_ff),
8             nn.ReLU(),
9             nn.Linear(d_ff, d_model)
10        )
11        self.norm1 = nn.LayerNorm(d_model)
12        self.norm2 = nn.LayerNorm(d_model)
13        self.dropout = nn.Dropout(dropout)
14
15    def forward(self, x):
16        # x: (seq_len, batch_size, d_model)
17        attn_output, _ = self.self_attn(x, x, x)
18        x = self.norm1(x + self.dropout(attn_output))
19        ffn_output = self.ffn(x)
20        x = self.norm2(x + self.dropout(ffn_output))
21        return x

```

Listing 4: Minimal Transformer Implmentation

Since the Transformer contains no recurrence or convolution, positional information must be injected explicitly. This is achieved using positional encodings, which are added to the input embeddings to provide the model with information about token order. A minimal Transformer encoder implemented in PyTorch is shown in Listing 4.

3 Sinusoidal Positional Encoding

Discuss the importance of positional encoding in Transformers. Implement sinusoidal positional encoding in PyTorch and demonstrate how it integrates with input embeddings. Explain why positional information is critical for sequence modeling.

Transformers process input sequences in parallel and do not contain any inherent notion of token order. As a result, positional information must be explicitly provided to the model in order to preserve the structure of sequential data. Without positional encoding, a Transformer would treat the input as a bag of tokens, making it incapable of distinguishing between different permutations of the same sequence. Positional encoding therefore plays a critical role in sequence modeling tasks such as machine translation, language modeling, and text summarization.

The original Transformer architecture introduces *sinusoidal positional encodings* [5],

which inject position-dependent signals into the input embeddings using deterministic functions. These encodings are added directly to the word embeddings, allowing the model to leverage both semantic and positional information simultaneously. Since the positional encodings have the same dimensionality as the embeddings, this integration does not increase the model complexity.

For a given position pos and embedding dimension i , the sinusoidal positional encoding is defined as

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (10)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (11)$$

where d_{model} denotes the embedding dimension. Sine functions are used for even indices and cosine functions for odd indices, resulting in a unique positional encoding for each position. This formulation ensures that positions that are close in the sequence have similar encodings, while still allowing the model to distinguish distant positions.

An important property of sinusoidal positional encodings is that they allow the model to generalize to sequence lengths longer than those seen during training. Since the encodings are generated using fixed mathematical functions rather than learned parameters, they can be computed for arbitrary sequence lengths without retraining. Additionally, relative positions can be expressed as linear functions of these encodings, which is beneficial for attention-based models.

In practice, positional encodings are added element-wise to the input embeddings before being passed to the Transformer encoder. If $E \in \mathbb{R}^{T \times d_{\text{model}}}$ denotes the token embeddings and $P \in \mathbb{R}^{T \times d_{\text{model}}}$ the positional encodings, the combined input is given by

$$X = E + P. \quad (12)$$

This simple operation allows every attention layer in the network to access positional information implicitly.

A PyTorch implementation of sinusoidal positional encoding is shown in Listing 5. The code constructs a positional encoding matrix once and registers it as a buffer, ensuring that it is not treated as a learnable parameter. This positional encoding is then added to the input embeddings at each forward pass, providing positional information into the Transformer pipeline.

4 Git Commit Log

```
9915a65 2025-12-19 Nakul Bhat build(flake): add nltk
3c8df5e 2025-12-19 Nakul Bhat feat(report): add transformers section
04f7210 2025-12-19 Nakul Bhat feat(report): add seq2seq section
dedfb6b 2025-12-19 Nakul Bhat feat(report/listings): add seq2seq model
037c3a3 2025-12-19 Nakul Bhat refactor(code): cleanup
a22e5b3 2025-12-10 Nakul Bhat feat(report): add git history
4635a3f 2025-12-10 Nakul Bhat feat(report): add second page
814ffcb 2025-12-10 Nakul Bhat feat(report): setup packages
```

```

1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model, max_len=5000):
3         super().__init__()
4         pe = torch.zeros(max_len, d_model)
5         position = torch.arange(0, max_len).unsqueeze(1)
6         div_term = torch.exp(
7             torch.arange(0, d_model, 2) * (-math.log(10000.0) /
8                 ↪ d_model))
9         pe[:, 0::2] = torch.sin(position * div_term)
10        pe[:, 1::2] = torch.cos(position * div_term)
11        pe = pe.unsqueeze(1) # (max_len, 1, d_model)
12        self.register_buffer("pe", pe)
13
14    def forward(self, x):
15        # x: (seq_len, batch_size, d_model)
16        x = x + self.pe[:x.size(0)]
17        return x

```

Listing 5: Sinusoidal Positional Encoding

```

9452fdd 2025-12-10 Nakul Bhat feat(report): add titlepage
3b209ac 2025-12-09 Nakul Bhat feat(code): add sentiment analysis pipeline
1b7ce83 2025-12-09 Nakul Bhat build(flake): add tqdm
8be83c8 2025-12-08 Nakul Bhat feat(seq2seq): add seq2seq model
bb49af7 2025-12-08 Nakul Bhat build(flake): add matplotlib and pandas
8fab023 2025-12-05 Nakul Bhat feat(code): add seq2seq loader
0008289 2025-12-05 Nakul Bhat chore(gitignore): ignore data dir
d3338b9 2025-12-04 Nakul Bhat build(flake): update flake to add cuda
f66d0e4 2025-12-03 Nakul Bhat feat(report): start sections/seq2seq
9aaf0c7 2025-12-03 Nakul Bhat build(flake): add flake for pytorch
74915b0 2025-12-03 Nakul Bhat chore(gitignore): add python gitignore
4f871cf 2025-11-27 Nakul Bhat feat(readme): add note for visitors
8a6177f 2025-11-27 Nakul Bhat Initial commit

```

References

- [1] Kyle Aitken et al. *Understanding How Encoder-Decoder Architectures Attend*. Oct. 28, 2021. DOI: 10.48550/arXiv.2110.15253. arXiv: 2110.15253 [cs]. URL: <http://arxiv.org/abs/2110.15253> (visited on 12/10/2025). Pre-published.
- [2] Dzmitry Bahdanau et al. “End-to-End Attention-Based Large Vocabulary Speech Recognition”. In: *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). Mar. 2016, pp. 4945–4949. DOI: 10.1109/ICASSP.

- 2016.7472618. URL: <https://ieeexplore.ieee.org/abstract/document/7472618> (visited on 12/19/2025).
- [3] Sean Robertson. *NLP From Scratch: Translation with a Sequence to Sequence Network and Attention — PyTorch Tutorials 2.9.0+cu128 Documentation*. URL: https://docs.pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html (visited on 12/10/2025).
- [4] *Tatoeba: Collection of Sentences and Translations*. URL: <https://tatoeba.org/en> (visited on 12/19/2025).
- [5] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html> (visited on 12/19/2025).